

P2P Engine

P2P Engine è un sistema di memoria progettuale per contesti nei quali persone e agenti di intelligenza artificiale devono condividere informazioni senza perdere motivazioni, vincoli e responsabilità delle scelte importanti. Mantiene questa memoria locale, ispezionabile e versionata, quindi ne deriva viste più compatte che aiutano ogni partecipante a recuperare la parte di progetto rilevante per il compito corrente.

Scopo e risultato atteso

L'obiettivo centrale è conservare l'intento progettuale come conoscenza durevole e verificabile. Le idee iniziali devono poter evolvere in un progetto coerente senza costringere una persona o un agente a rileggere ogni artefatto storico prima di compiere il passo successivo.

Il motore è progettato attorno a file locali e alla cronologia Git. Questa scelta rende la conoscenza portabile e revisionabile, permettendo a strumenti deterministici di validarla, sintetizzarla e recuperarla. Il risultato atteso non è un responsabile di progetto autonomo: è uno strato affidabile di memoria e mediazione che aiuta le persone responsabili a decidere e fornisce agli agenti un contesto circoscritto per lavorare in modo utile.

Persone, agenti e autorità

Gli utenti principali sono proprietari di progetto, sviluppatori individuali, piccoli gruppi tecnici, manutentori e agenti di programmazione o pianificazione. Il proprietario resta responsabile delle decisioni che attribuiscono o interrompono l'autorità progettuale. Gli agenti possono analizzare evidenze, proporre alternative, preparare candidati ed eseguire operazioni esplicitamente consentite, ma non decidono silenziosamente al posto del proprietario.

La distinzione è incorporata nelle superfici del prodotto. Le analisi in sola lettura possono restare leggere. Le modifiche persistenti usano comandi dichiarati, controlli dei permessi e, quando l'impatto lo richiede, un'anteprima seguita da conferma esplicita. Le istruzioni generate adattano questi confini agli ambienti agentici supportati mantenendo una base comune.

Dall'intento alla memoria progettuale operativa

Un flusso tipico parte da un'idea, la esplora e la confronta con la conoscenza già presente. La proposta risultante registra problema, risultato desiderato, alternative,

rischi e condizioni di accettazione. Scelte e conflitti diventano espliciti invece di restare nascosti nella conversazione.

Quando viene scelta una direzione, il sistema può derivare descrizioni operative del cambiamento, metadati di lavorazione e specifiche software. Questi artefatti collegano progettazione ed esecuzione, ma non dimostrano che un'implementazione esterna sia avvenuta. Se una direzione precedente non è più adatta, la sua storia resta disponibile e la relativa autorità viene modificata con eventi espliciti, senza cancellare il record precedente.

Come è organizzata la memoria del progetto

I documenti Markdown destinati alle persone e i dati YAML strutturati conservano il record durevole del progetto. Attorno a questo record il motore costruisce registri, una vista verticale del progetto, contesto decisionale, indicatori di avanzamento e azioni successive ordinate. Questi derivati sono rigenerabili e non possono trasformarsi in fonti concorrenti.

Il recupero delle informazioni deve essere circoscritto e spiegabile. Una richiesta dovrebbe ricevere decisioni vicine, vincoli, relazioni ed evidenze pertinenti invece di un prefisso generico dell'archivio delle proposte. Provenienza, autorità, completezza e freschezza restano collegate al risultato, affinché la compressione non nasconda l'incertezza e un'associazione euristica non diventi una decisione.

Interfacce e integrazioni

La riga di comando è la superficie locale di riferimento per operare sul workspace. Un server MCP locale espone strumenti specifici del dominio agli agenti compatibili e mantiene esplicite le operazioni privilegiate o non disponibili, invece di simularle. Le istruzioni generate per gli adattatori consentono a diversi client agentici di convivere nello stesso repository.

Git fornisce cronologia durevole e collaborazione gestita facoltativa. Gli ambienti Python locali al progetto e gli artefatti di rilascio versionati forniscono il runtime. Gli strumenti per le specifiche e i renderer di pubblicazione consumano output derivati. Le operazioni sui fornitori esterni restano limitate dalle capacità disponibili e subordinate ai permessi.

Verticale e completezza del progetto

Ogni progetto seleziona un verticale che descrive le domande essenziali e le sezioni necessarie nel proprio dominio. Per un progetto software questa lente comprende scopo, utenti, perimetro, flussi, dati, integrazioni, vincoli di qualità, validazione, rischi e decisioni importanti. Il verticale orienta la completezza, ma non impone un indice rigido a ogni documento.

Le evidenze della definizione e quelle delle proposte restano distinte. Il motore può suggerire relazioni probabili, ma solo le evidenze dichiarate hanno autorità. Le lacune diventano domande strutturate per il proprietario e le risposte producono modifiche candidate alla definizione, che devono essere visualizzate e confermate prima di cambiare il record progettuale.

Specifiche, consegne e pubblicazione

Le specifiche software vengono generate dalla direzione governata del progetto e dal perimetro del cambiamento collegato. Sono consegne per l'implementazione: gruppi o strumenti esterni possono usarle per costruire il software, mentre P2P Engine resta responsabile della qualità e della tracciabilità della conoscenza progettuale senza dichiarare che l'implementazione sia stata completata.

L'esportazione visibile fornisce il materiale completo per la ricerca. La pubblicazione per le persone aggiunge un modello editoriale che riorganizza le evidenze per un lettore che non conosce il flusso a monte. Curatela, validazione, rendering PDF e revisione del proprietario restano fasi separate, così un file ben presentato non implica da solo un'approvazione.

Runtime, schema e modifiche sicure

La compatibilità del runtime e quella dello schema del workspace sono indipendenti. Un progetto dichiara l'intervallo compatibile del motore e il runtime consigliato, mentre il workspace dichiara la versione della propria struttura dati. La base Python supportata è la versione 3.11 o successiva.

Le transizioni di schema sono esplicite e procedono in avanti. La pianificazione della migrazione è in sola lettura; l'applicazione verifica il piano revisionato e lo stato atteso, conserva per impostazione predefinita il materiale sconosciuto e rende disponibile il recupero in caso di interruzione. Le modifiche su più file devono validare candidati completi ed essere applicate atomicamente. Le operazioni di stato e contesto in sola lettura devono restare deterministiche e prive di scritture persistenti.

Confini, ipotesi e rischi

Il motore locale comprende inizializzazione del progetto, memoria governata, supporto alle decisioni, definizione verticale, recupero delle informazioni, validazione, integrazione con gli agenti, specifiche ed esportazioni. Servizio multi-tenant ospitato, decisioni autonome al posto del proprietario, mutazioni non circoscritte su cloud o fornitori Git, modifiche arbitrarie allo stato governato e distribuzione garantita tramite registri pubblici restano fuori dal perimetro.

La progettazione presuppone spazio locale scrivibile, cronologia Git, runtime compatibile e un proprietario responsabile disponibile per le decisioni semantiche. L'adozione in un ambiente operativo differente deve verificare queste ipotesi.

I rischi principali sono viste derivate obsolete, scritture parziali, variazioni concorrenti delle fonti, migrazioni automatiche che cambiano significato, evidenze euristiche scambiate per autorità e pubblicazioni generate interpretate come revisionate. Il progetto li contrasta con impronte digitali, controlli di freschezza, provenienza circoscritta, autorità esplicita, transazioni atomiche, migrazione conservativa e separazione tra curatela e revisione. Questi controlli riducono il rischio, ma non giustificano la rimozione delle incertezze residue.

Come si riconosce il successo

Un workspace efficace può essere inizializzato, ispezionato e migrato tramite interfacce supportate. Le persone possono passare dall'intento a una definizione revisionabile, recuperare il contesto pertinente, preparare specifiche per strumenti esterni e validare lo stato risultante senza riparare manualmente i file governati.

Il sistema deve rendere visibile l'incertezza, mantenere esplicite le azioni controllate dal proprietario, conservare le motivazioni storiche e consentire la ricostruzione di ogni vista derivata a partire da evidenze durevoli. I controlli di compatibilità e le suite di test mirate e complete forniscono evidenza implementativa della tenuta di questi contratti.

Contributi

La sintesi corrente si basa su 204 contributi registrati esplicitamente.

Contributore	Quota registrata
local	44.12%
codex	43.63%
davide-via-codex	2.94%
owner	2.94%
owner-via-codex	2.94%
bootstrap	2.45%
intake:INTAKE-001	0.49%
intake:INTAKE-002	0.49%

Le percentuali rappresentano quote dei contributi registrati esplicitamente; non misurano impegno, qualità, merito, proprietà, paternità del codice o proprietà intellettuale.